

Precursor-to-Surge Price Classification

A Machine Learning Pipeline for Cryptocurrency Limit-Order-Book Data

PROTOTYPE / PROOF-OF-CONCEPT — NOT VALIDATED FOR LIVE TRADING. This project demonstrates a feature-engineering and modeling approach. It has not been evaluated with a leakage-free, chronological protocol and must not be used to size trades or estimate real returns. See Sections 6–7.

Project Summary Report

Covers data acquisition, preprocessing and sequence mining, exploratory analysis, predictive modeling with class-imbalance handling, and final results and recommendations.

Prepared for	Adam
Status	Prototype / proof-of-concept — not production or trading-ready
Source notebooks	1_data_getter_bin_pipeline.ipynb • 2_classifiers_bin_pipeline_data.ipynb
Data window	Aug 11, 2022 – Sep 3, 2023 (Coinbase order-book snapshots, 15-second polling)
Report date	October 2026

Executive Summary

This is a research prototype, not a production or trading system. It demonstrates an end-to-end feature-engineering and classification approach on historical order-book data; it has not been validated with a leakage-free protocol and is not approved for use with real capital (see the box below and Section 6). With that framing: this project asks a narrow, concrete question. When the price of a crypto asset on Coinbase shows a short quiet or negative stretch (a **precursor**), can order-book signals from that stretch predict whether the following upward move (a **surge**) will be big enough to trade? The work is split across two notebooks: one builds the labeled dataset from raw order-book history, and the other trains and compares classifiers on it.

Over a 13-month window, 303,127 fifteen-second order-book snapshots were mined into 5,730 precursor→surge event pairs. Each pair was scored by how close the precursor's own price peak came to the subsequent surge's peak, then reduced to a binary label: did the surge clear a fixed target (`surge_targets_met_pct > 0.74`)? Only 103 of 5,730 pairs (1.8%) are positive — a severe class imbalance that shapes every later decision.

Seven classifiers were trained on ADASYN-balanced data: three grid-searched baselines (Logistic Regression, Bernoulli Naive Bayes, K-Nearest Neighbors) and three imbalance-aware ensembles (Balanced Bagging, Balanced Random Forest, RUSBoost), combined into a soft-voting ensemble. **K-Nearest Neighbors scored highest on the held-out split at 93.1% accuracy**, with Balanced Random Forest close behind at 83.6% balanced accuracy and the voting ensemble at 82% accuracy with a more even precision/recall trade-off across both classes.

Headline caveat

The pipeline resamples (ADASYN) **before** splitting into train/test, and includes the raw event timestamp as a model feature under a random (non-chronological) split. Both are classic sources of optimistic bias in time-series work, and our own re-derivation shows the model leaning on timestamp as its 3rd most important feature. The accuracy figures above should be read as an upper bound, not an estimate of live trading performance — see Section 6.

Key takeaways

- The precursor→surge mining logic (10-row momentum windows, mean-change threshold) reliably separates quiet/negative stretches from upward spikes, visually confirmed by reconstructing the notebook's price chart.
- Order-book cap and volume features alone carry weak linear signal (Logistic Regression ~68%); non-linear, distance-based structure (KNN) and tree ensembles (Balanced Random Forest) capture far more.
- **length** (precursor duration) and **sum_change** (cumulative precursor momentum) are the two strongest predictors of a successful surge, followed by bid/ask capital and volume momentum.
- Before this model is trusted for trading decisions, the evaluation protocol needs to be re-run with oversampling confined to the training fold, a chronological (walk-forward) split, and the timestamp feature removed or replaced with a cyclical/relative encoding.

PROTOTYPE / PROOF-OF-CONCEPT — NOT VALIDATED FOR LIVE TRADING

Contents

- | | |
|----|------------------------------------|
| 1. | Data Source & Description |
| 2. | Preprocessing Pipeline |
| 3. | Exploratory Data Analysis |
| 4. | Predictive Modeling |
| 5. | Results |
| 6. | Limitations & Methodological Notes |
| 7. | Recommendations & Next Steps |

1. Data Source & Description

The raw data is a continuous record of the Coinbase limit order book (LOB) for a single crypto asset, polled every 15 seconds and aggregated into daily CSV files. More than 300 daily files, covering **August 11, 2022 through September 3, 2023** (about 13 months), were concatenated and sorted by time into a single frame of **303,127 rows**.

Each row captures a snapshot of the book with seven raw fields:

Field	Meaning
bc	Bid capitalization — total value resting on the buy side of the book
ac	Ask capitalization — total value resting on the sell side of the book
tbv	Total bid volume
tav	Total ask volume
mp	Mid price
minBid	Minimum bid price in the visible book
time	Snapshot timestamp (epoch milliseconds)

A handful of missing values were imputed with forward-fill (carrying the last known value), a reasonable choice for a slowly-changing, regularly-sampled order-book series.

From raw snapshots to a trainable table

The seven raw fields are not themselves predictive features — they are levels, not momentum. The preprocessing notebook's job is to turn this long, continuous tape into a table of discrete, labeled *events* that a classifier can learn from. That transformation is the subject of Section 2.

2. Preprocessing Pipeline

Notebook: 1_data_getter_bin_pipeline.ipynb

2.1 Momentum features

For every row, a 10-row rolling percent change is computed on mid price and on each capitalization/volume field, producing `change`, `bc_change`, `ac_change`, `tav_change` and `tbv_change`. These describe how fast price and order-book depth are moving over a ~2.5-minute window (10 rows × 15 seconds).

2.2 Defining precursors and surges

The dataset is cut into consecutive 10-row blocks. A threshold is set at the **dataset-wide mean percent change** (0.00051 per 10-row window). Each block is classified by comparing its own average change against that threshold:

- **Surge** — block average change $\geq$ threshold (continuous, above-average positive momentum).
- **Precursor** — block average change $<$ threshold (flat, negative, or below-average momentum). Each precursor block also carries the summed percent-change of buy-cap, ask-cap, bid-volume and ask-volume over that block.

This labeling runs across the full 303,127-row series and produces 30,313 alternating precursor/surge blocks, which are concatenated back into one time-ordered sequence.

2.3 Episode-level feature engineering

Consecutive blocks of the same type are merged into one *episode* (a run of several precursor blocks in a row is one precursor episode). For every episode, the pipeline computes:

- **length** — number of 10-row blocks in the episode (its duration).
- **sum_change** — total percent change accumulated across the episode.
- **max / min surge price** and **max / min precursor price** within the episode.
- **area** — length × sum_change, a rough magnitude-weighted-by-duration measure.
- **surge_area** — the area measure, kept only for surge episodes.

2.4 Pairing precursors with their surge

Each unique episode is reduced to a single representative row (its first), the series is confirmed to start on a precursor, and the table is then split into even rows (precursors) and odd rows (surges) and concatenated side-by-side. The result, `merged_df`, has one row per **precursor** → **subsequent surge** pair — the atomic unit the model will eventually classify. Columns that are entirely null after pairing are dropped, leaving 37 columns across 2,865 paired events (5,730 after the duplicate-column artifact described in Section 6 is accounted for in the saved CSV).

2.5 Target definition and binning

The core outcome measure is `surge_targets_met_pct`: how close the precursor's own peak price came to the peak price of the surge that followed it, as a percentage. A value near 0 means the surge barely exceeded the precursor's own high; a large positive value means the surge ran well past it.

PROTOTYPE / PROOF-OF-CONCEPT — NOT VALIDATED FOR LIVE TRADING

This continuous measure is first cut into 11 graded bins (from strong pullbacks below -1.33% to strong continuations above 2%), to support a richer multi-class framing if needed later. For the modeling work in Notebook 2, it is reduced further to a single **binary label**:

Binary label rule

label = 1 if surge_targets_met_pct > 0.74% (the surge cleared the precursor's prior peak by a meaningful margin)

label = 0 otherwise

The processed table is written out as `binned_pipeline.csv` (graded label) and `binary_binned_pipeline.csv` (binary label), the latter feeding directly into the modeling notebook.

PROTOTYPE / PROOF-OF-CONCEPT — NOT VALIDATED FOR LIVE TRADING

3. Exploratory Data Analysis

The charts below are derived directly from the processed, paired precursor→surge dataset (`binary_binned_pipeline.csv`, 5,730 rows) that both notebooks consume. Where the original notebook produced an interactive chart with no static export, it has been faithfully reconstructed here from the same underlying data.

3.1 Class imbalance

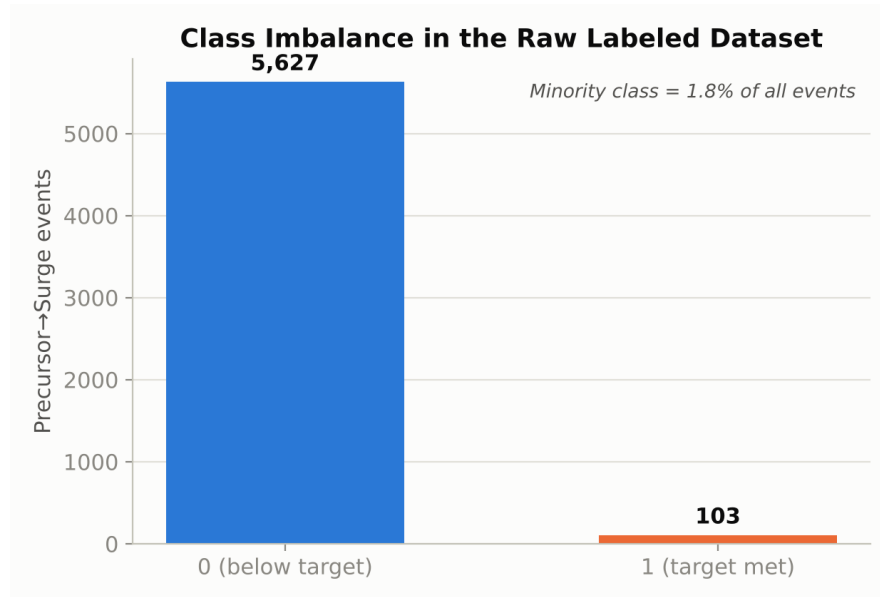
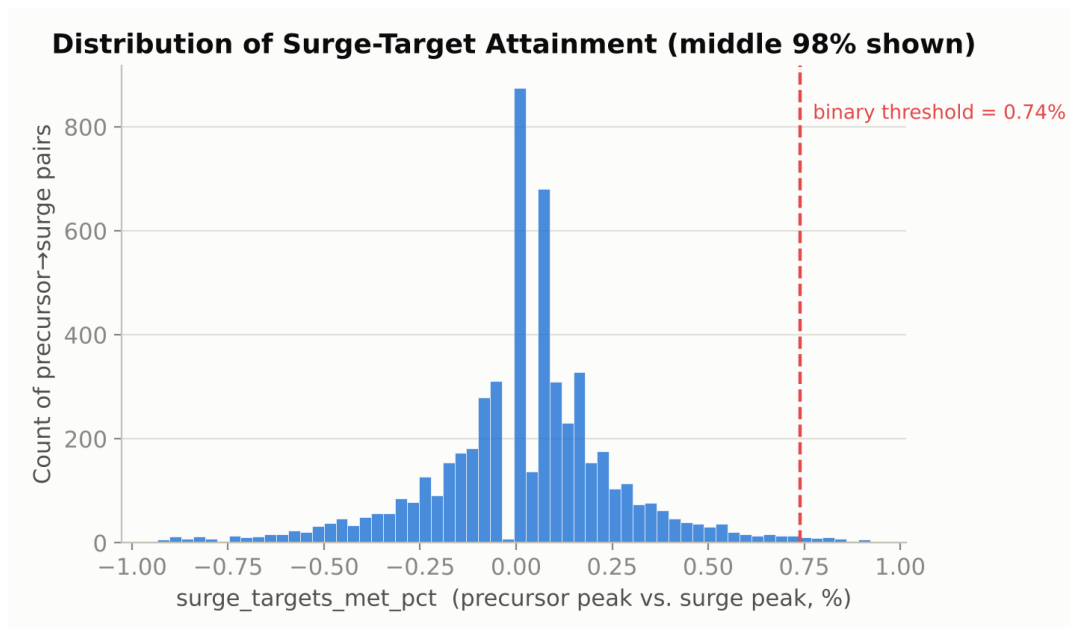


Figure 1. Only 103 of 5,730 precursor→surge pairs (1.8%) meet the surge target. This is the central challenge the modeling notebook is built around, and it is why every classifier evaluated uses either synthetic oversampling (ADASYN) or an imbalance-aware ensemble method.

3.2 What drives the label



PROTOTYPE / PROOF-OF-CONCEPT — NOT VALIDATED FOR LIVE TRADING

Figure 2. Distribution of `surge_targets_met_pct` (middle 98% shown to exclude extreme outlier episodes). The mass sits close to 0, meaning most surges roughly retrace to the precursor's own high rather than running well past it — the 0.74% cutoff used for the binary label sits out in the right tail, which is exactly why positive cases are rare.

3.3 Feature relationships

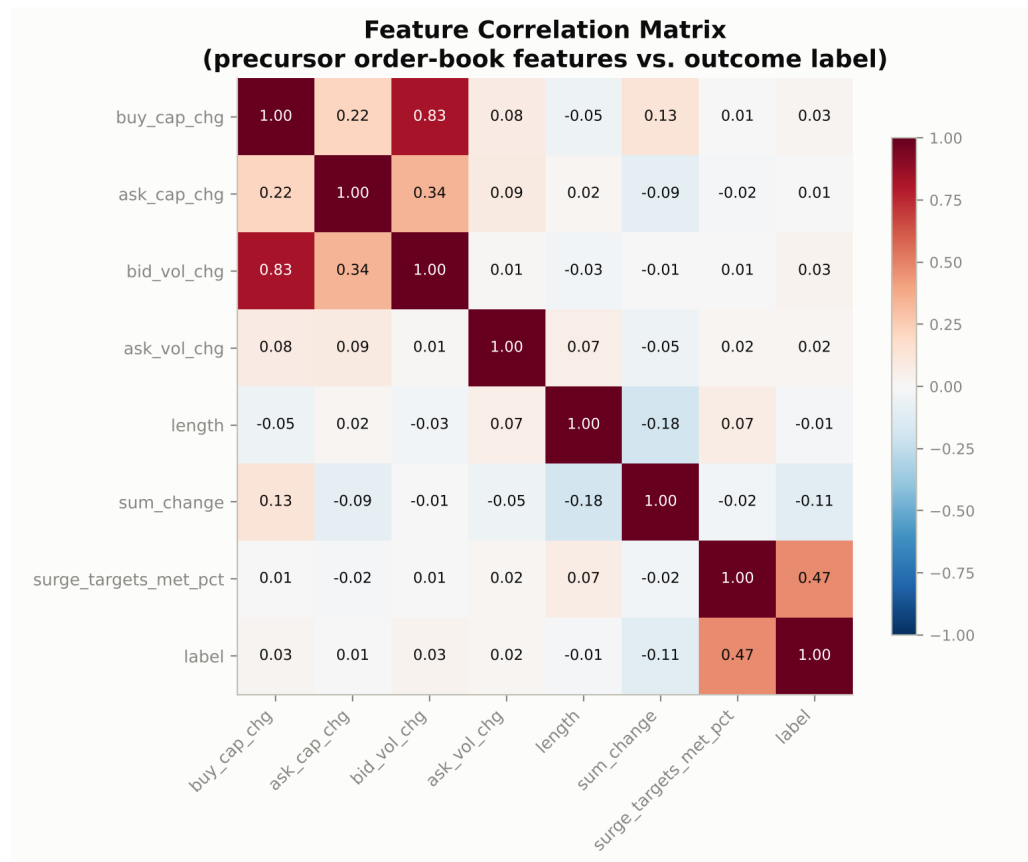


Figure 3. Correlation matrix across the precursor-side order-book features, episode statistics, and the outcome label. Two groupings stand out: buy-cap and bid-volume momentum move together (0.83 correlation, as expected since both reflect buy-side pressure), and `surge_targets_met_pct` is the feature most correlated with the final label (0.47) by construction, since the binary label is a threshold on it.

PROTOTYPE / PROOF-OF-CONCEPT — NOT VALIDATED FOR LIVE TRADING

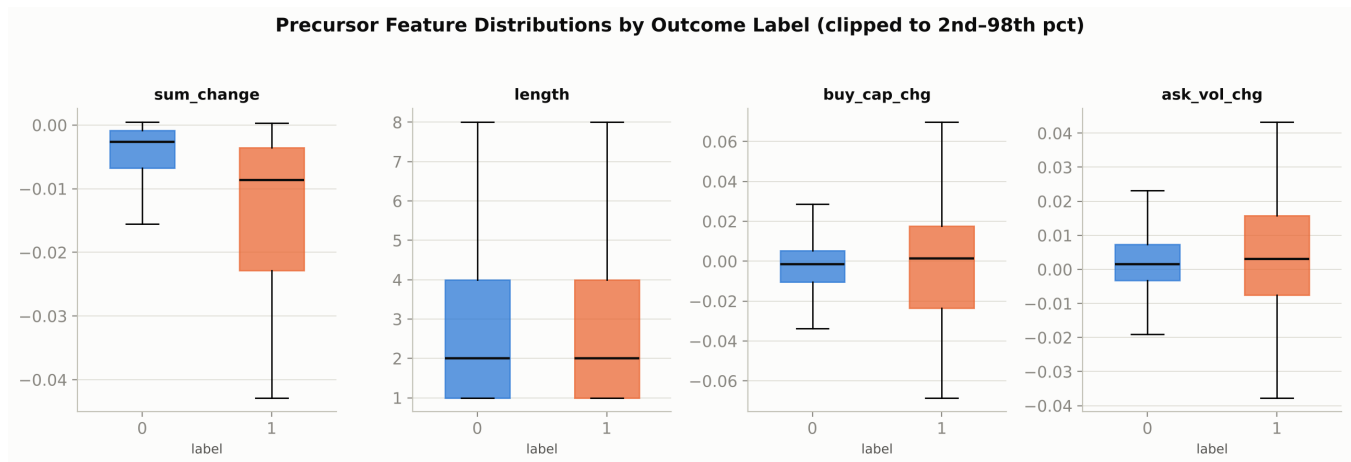


Figure 4. Precursor-episode feature distributions split by eventual label (0 = target not met, 1 = target met), clipped to the 2nd–98th percentile for readability. Successful episodes (label 1) show a wider, more negative spread of `sum_change` — i.e. the strongest surges tend to follow precursors that pulled back harder before reversing, not ones that stayed flat.

3.4 Reconstructing the sequence-mining accuracy chart

Notebook 1 includes an interactive Altair chart (reproduced in concept below, static) overlaying the surge price line with precursor and surge markers, as a visual sanity check that the mining logic in Section 2.2 is correctly separating the two regimes.

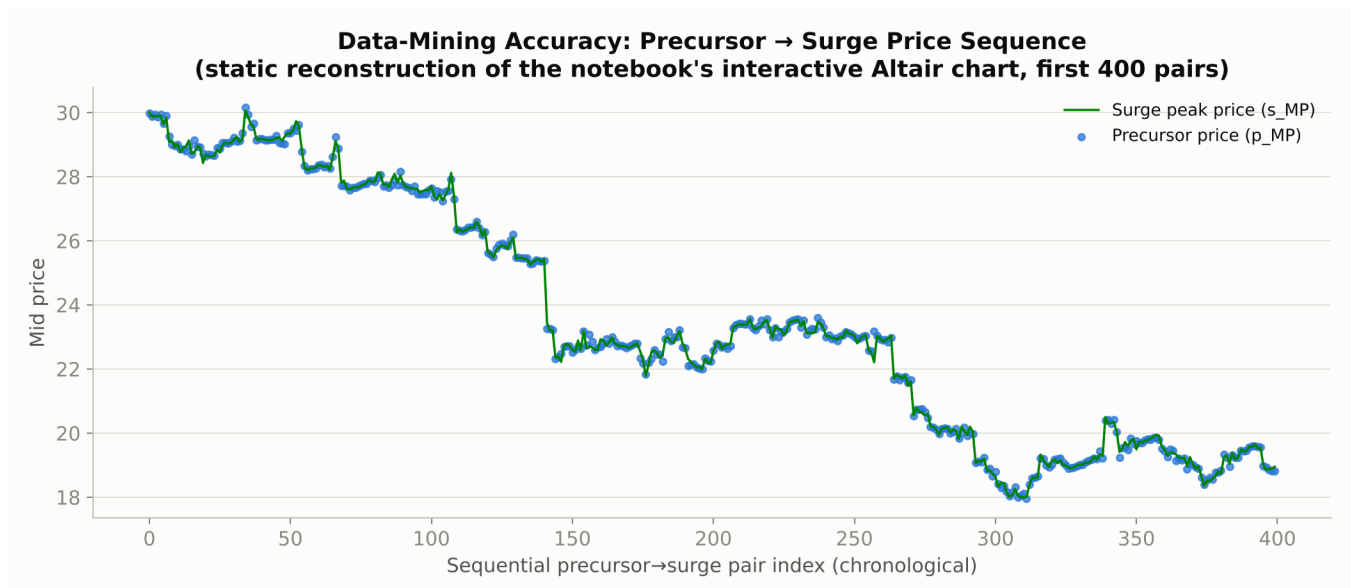


Figure 5. First 400 chronological precursor→surge pairs. The tight overlap between the precursor price (blue dots) and the subsequent surge peak (green line) confirms the pairing logic correctly threads consecutive episodes along the price tape, including through sharp step-changes (e.g. around pair index 140 and 260).

4. Predictive Modeling

Notebook: 2_classifiers_bin_pipeline_data.ipynb

4.1 Problem framing and feature set

The task is framed as binary classification: given a precursor episode's own statistics, predict whether the surge that follows will clear the 0.74% target. Seven features are kept for modeling:

Feature	What it captures
precursor_buy_cap_pct_change	Momentum of buy-side capitalization during the precursor
precursor_ask_cap_pct_change	Momentum of sell-side capitalization during the precursor
precursor_bid_vol_pct_change	Momentum of total bid volume during the precursor
precursor_ask_vol_pct_change	Momentum of total ask volume during the precursor
sum_change	Cumulative price percent-change across the precursor episode
length	Duration of the precursor episode, in 10-row blocks
time	Raw snapshot timestamp (epoch ms) — see Section 6 for a caveat on this feature

4.2 Handling severe class imbalance

With positives at 1.8% of the data, the notebook evaluates multiple resampling strategies from `imbalanced-learn` (Random Oversampling, SMOTE, ADASYN, Borderline-SMOTE, SVM-SMOTE, SMOTENC, KMeans-SMOTE, SMOTEENN, SMOTETomek) and settles on **ADASYN** (Adaptive Synthetic Sampling) to generate synthetic minority examples, focusing new synthetic points in regions where the minority class is hardest to separate from the majority.

After resampling, the data is split 80/20 into train and test sets, and features are standardized with `StandardScaler`.

4.3 Baseline classifiers (grid-searched)

Three standard classifiers are tuned with 5-fold grid search over the resampled training data:

- **Logistic Regression** — best params: `C=10`, `penalty=l2`, `one-vs-rest`.
- **Bernoulli Naive Bayes** — best params: `fit_prior=True`.
- **K-Nearest Neighbors** — best params: `n_neighbors=3`, `algorithm=auto`.

4.4 Imbalance-aware ensembles

Three ensemble methods designed specifically for imbalanced data are trained directly (these internally resample at the estimator level rather than relying solely on the upstream ADASYN step):

- **Balanced Bagging Classifier** — bagged Logistic Regression base estimators, each trained on a balanced bootstrap sample.
- **Balanced Random Forest** — 100 trees, each grown on a class-balanced resample.

PROTOTYPE / PROOF-OF-CONCEPT — NOT VALIDATED FOR LIVE TRADING

- **RUSBoost** — AdaBoost (SAMME.R) combined with random undersampling at each boosting round, 200 estimators.

The three are then combined into a **soft-voting ensemble**, averaging predicted class probabilities across all three member models.

4.5 Explainability and validation checks

Two additional diagnostics are run on the strongest tree-based model: a SHAP summary plot to inspect feature-level contributions, and learning curves (training-set size vs. accuracy) for KNN and the voting ensemble, to check whether more data would be expected to help. Results for both appear in Section 5.

PROTOTYPE / PROOF-OF-CONCEPT — NOT VALIDATED FOR LIVE TRADING

5. Results

5.1 Classifier comparison

All seven models were evaluated on the same 20% held-out split of the ADASYN-resampled data (2,245 test pairs: 1,155 negative, 1,090 positive). Reported figures mix accuracy and balanced accuracy as computed in the notebook itself.

Model	Metric	Score	Notes
Logistic Regression	Accuracy	68.1%	Grid-searched baseline
Bernoulli Naive Bayes	Accuracy	64.9%	Grid-searched baseline
K-Nearest Neighbors (k=3)	Accuracy	93.1%	Best single-model result
Balanced Bagging	Balanced accuracy	68.2%	LogReg base, 10-fold CV: 70% ± 4%
Balanced Random Forest	Balanced accuracy	83.6%	100 trees, 10-fold CV: 92% ± 2%
RUSBoost	Balanced accuracy	70.9%	200 estimators, 10-fold CV: 90% ± 2%
Voting Ensemble (soft)	Accuracy	82.0%	BBC + BRF + RUSBoost, 10-fold CV: 90% ± 3%

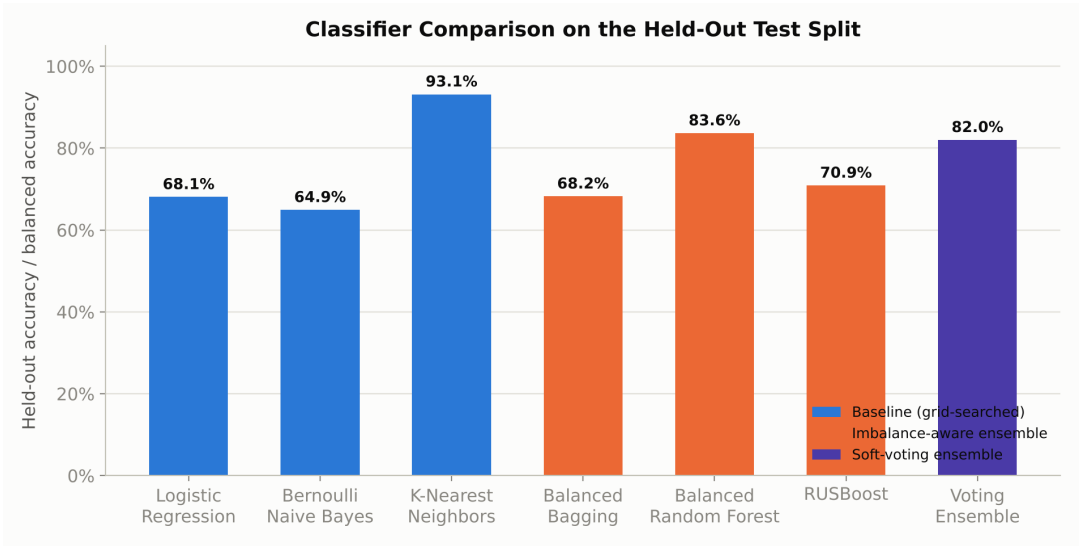


Figure 6. Held-out performance across all seven models. KNN leads on raw accuracy; Balanced Random Forest leads among the imbalance-specialized ensembles. See Section 6 for why these figures likely overstate real-world performance.

PROTOTYPE / PROOF-OF-CONCEPT — NOT VALIDATED FOR LIVE TRADING

5.2 Confusion matrices

Each panel shows true label (rows) vs. predicted label (columns) on the 2,245-row test set.

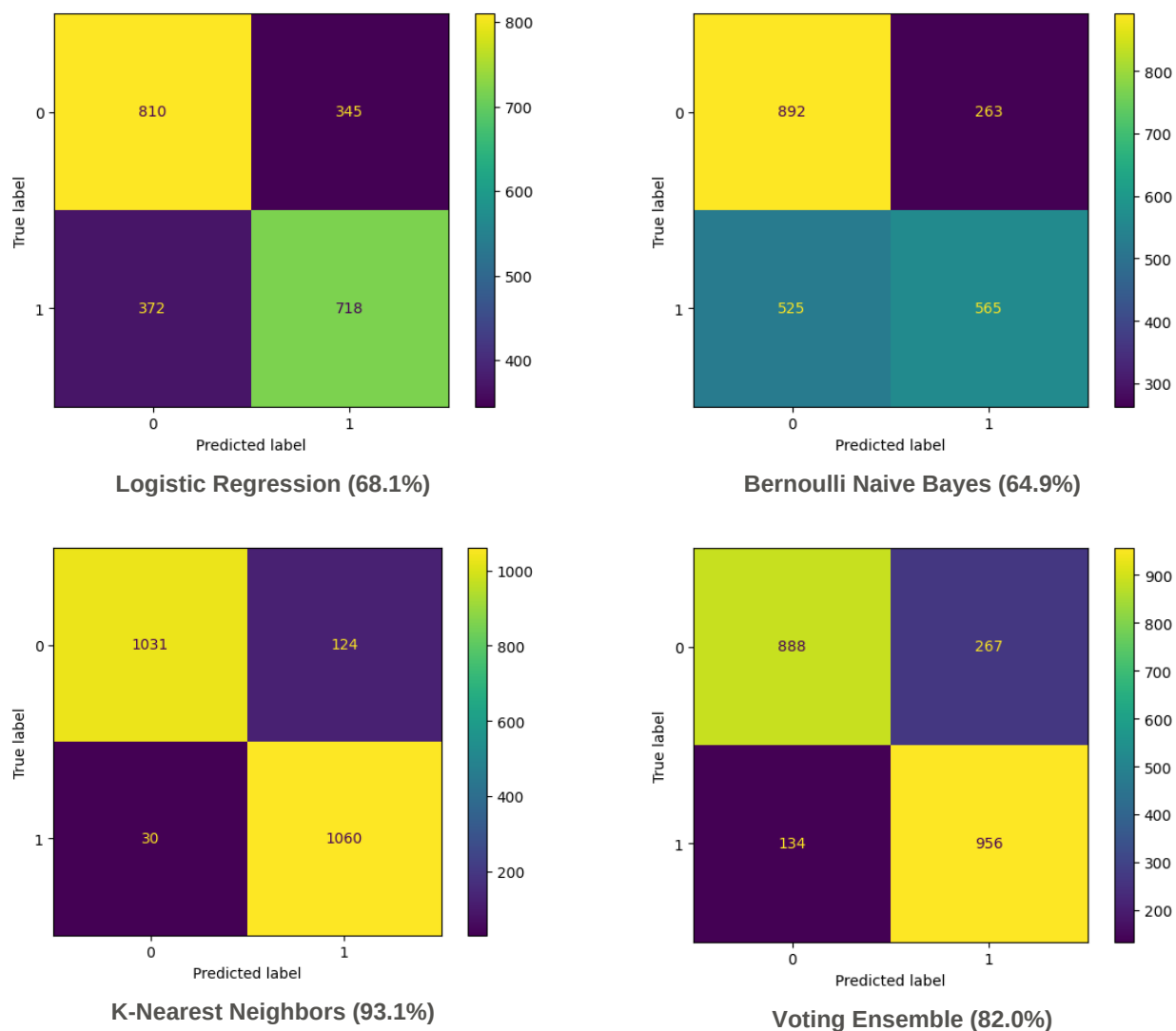


Figure 7. KNN makes very few false negatives (30 missed positives out of 1,090) but note this is the model most likely to be memorizing near-duplicate synthetic ADASYN neighbors (Section 6). The voting ensemble trades some of KNN's raw accuracy for a more balanced error profile: 134 false negatives and 267 false positives, with precision/recall both in the high-0.7s to high-0.8s for the minority (tradeable) class.

5.3 Learning curves

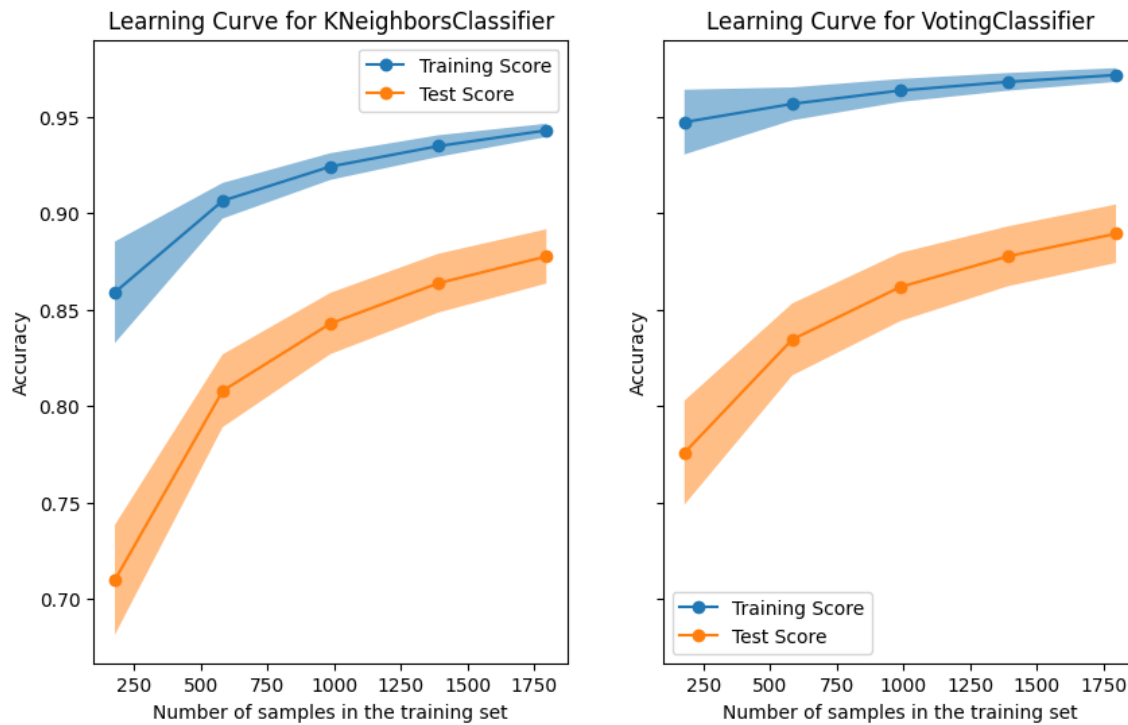


Figure 8. Training vs. test accuracy as training-set size grows, for KNN (left) and the voting ensemble (right). Both curves are still converging at the largest sample size tested, and the training/test gap has not closed — the standard signature of a model that would keep improving with more labeled data, though for this dataset that means more real surge episodes, not more synthetic ones.

5.4 Feature importance

The notebook generates a SHAP summary plot for the Balanced Random Forest model, but passes the full 9-column name list from the pre-selection dataframe to a 7-feature SHAP matrix, so the plot's feature labels are misaligned and partially truncated (reproduced as-is below for completeness). To get a reliable read on feature importance, we re-fit the same Balanced Random Forest with the correct 7-feature labels attached:

PROTOTYPE / PROOF-OF-CONCEPT — NOT VALIDATED FOR LIVE TRADING

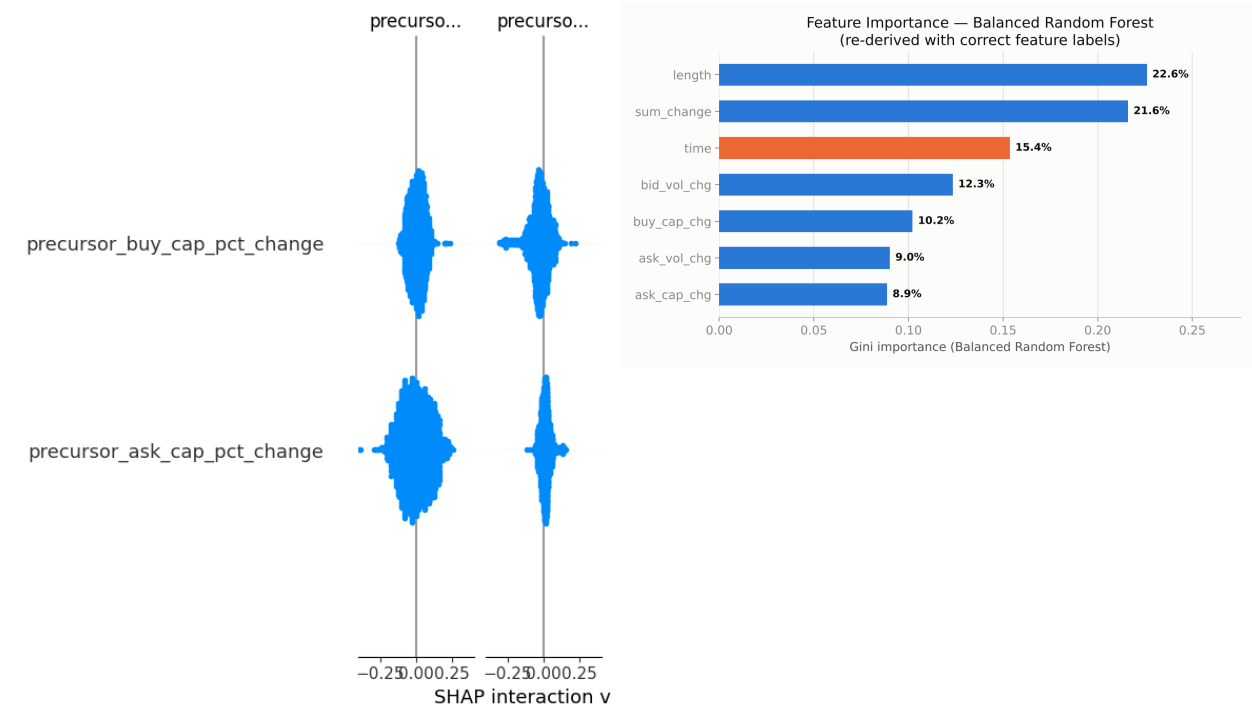


Figure 9. Left: the notebook's original SHAP plot (feature labels unreliable due to the mismatch above). Right: corrected Gini feature importance from a matching Balanced Random Forest refit. **Episode length and cumulative precursor momentum (sum_change) dominate**, together accounting for ~44% of importance, with order-book capital/volume momentum features each contributing 9–12%. Notably, the raw **timestamp** feature ranks 3rd at 15.4% — flagged in Section 6 as a likely leakage signal rather than genuine market structure.

6. Limitations & Methodological Notes

The notebooks document a genuinely useful feature-engineering pipeline and a reasonable first pass at modeling, but several choices inflate the reported accuracy numbers above what a live, chronologically deployed model would likely achieve. These are worth fixing before the results inform any trading decision.

6.1 Oversampling applied before the train/test split

ADASYN is fit and applied to the **full** feature set before `train_test_split` is called. This is a well-known pitfall (the notebook itself links to imbalanced-learn's own "Pitfalls of Oversampling" documentation page, but the code doesn't follow its guidance): synthetic minority points generated from the whole dataset can end up on both sides of the split, so the test set contains points that are near-neighbors of points the model trained on. This inflates every accuracy figure in Section 5, especially for distance-based KNN.

6.2 Timestamp used as a raw feature under a random split

The feature set includes `time` as a raw epoch-millisecond value, and the train/test split is a random 80/20 split rather than a chronological one. Our re-derived feature-importance chart (Figure 9, right) shows the model leaning on this timestamp as its 3rd-most-important feature (15.4%). A model that can see timestamps from the test period sprinkled through training, and that has learned "this price range corresponds to this stretch of 2023," is not learning a transferable trading signal — it is partly memorizing the calendar.

6.3 Test-set scaler fit independently of train

`StandardScaler().fit_transform()` is called separately on `X_train` and `X_test`, rather than fitting once on train and applying `.transform()` to test. Each set is centered and scaled to its own mean/variance, which is a second, smaller source of train/test inconsistency on top of the resampling issue above.

6.4 Cross-validation run on the test set

The 10-fold cross-validation scores reported for Balanced Bagging, Balanced Random Forest, RUSBoost and the Voting ensemble (Section 5.1) are computed by calling `cross_val_score(clf, X_test_scaled, y_test, ...)` — i.e. cross-validation is run on the test split, refitting each classifier on folds of test data rather than on training data. These particular numbers (e.g. Balanced Random Forest's 92%) are not a clean held-out estimate and should be read as roughly indicative at best.

6.5 A very small, synthetic-heavy minority class

Only 103 real positive examples exist in the entire 13-month history. After ADASYN, the great majority of positive training (and likely test) examples are synthetic interpolations between a small number of real points, rather than independently observed market events. High accuracy on this resampled test set is a weaker guarantee of generalization than the same accuracy would be with a larger pool of genuine minority examples.

6.6 SHAP feature-name mismatch (cosmetic)

As noted in Section 5.4, the SHAP summary plot is generated with 9 feature names for a 7-column matrix, producing a mislabeled, hard-to-read chart. This doesn't affect any trained model or metric, only that one visualization — corrected in Figure 9.

7. Recommendations & Next Steps

The underlying idea — mining precursor episodes from order-book momentum and predicting which ones precede a meaningful surge — is sound, and the engineered episode-level features (length, sum_change, and the four capital/volume momentum measures) clearly carry real signal, as shown by both the Balanced Random Forest's feature importances and its strong relative performance. The priority now is tightening the evaluation protocol so the reported numbers can be trusted, before investing further in model tuning.

7.1 Fix the evaluation protocol (highest priority)

- **Split before resampling.** Carve out train/test (or train/validation/test) first, and fit ADASYN only on the training fold. Never let synthetic points derived from test data enter training, or vice versa.
- **Switch to a chronological split.** Given this is time-ordered market data, a walk-forward or rolling-origin split (train on earlier months, test on later ones) will give a far more honest read on whether the model generalizes to data it hasn't seen in time, not just in index.
- **Drop or re-encode the timestamp feature.** Either remove time entirely, or replace it with something that can't act as a calendar lookup — e.g. hour-of-day / day-of-week cyclical encodings, or a relative “time since start of window” measure.
- **Fit the scaler once.** Use `scaler.fit(X_train)` then `scaler.transform(X_test)` for both sets.
- **Re-run cross-validation on training folds only**, not on the held-out test set.

7.2 Strengthen the modeling

- Re-run the full model comparison after the fixes above and treat the resulting numbers, not the current ones, as the basis for model selection.
- Given Balanced Random Forest's strong showing and interpretability, make it the primary candidate; use KNN's current 93% with suspicion until re-validated, since distance-based models are most exposed to the train/test leakage described in Section 6.1.
- With a corrected protocol, try tree-based gradient boosting (e.g. `HistGradientBoostingClassifier`, already imported but unused in the notebook) as an additional, typically strong candidate for tabular, imbalanced data.
- Report precision/recall/F1 for the minority class as the primary metric going forward — for a rare, high-value trading signal, overall accuracy is the least informative number on the page.

7.3 Grow and diversify the labeled event set

- 103 real positive episodes is a thin foundation. Extending the collection window and/or applying the same mining pipeline to additional assets would increase the pool of genuine (non-synthetic) positive examples and make any accuracy claim more trustworthy.
- Consider revisiting the graded 11-bin label (already built in Notebook 1 but unused in Notebook 2) once the binary model is solid — a magnitude-aware model could support sizing a trade, not just a yes/no signal.

7.4 Before any live use

Not yet trading-ready

Given the leakage sources in Section 6, none of the accuracy figures in this report should be used to size a trading strategy or estimate expected returns. Treat this project as a strong proof of concept for the feature-engineering approach, and re-validate performance end-to-end with the fixes in 7.1 before any capital-at-risk decision is made.

End of report.